

Style: unified avatars, one-size-fits-all wardrobe, Style Studio, AI item maker, Marketplace publishing

Status: DECISIONS SETTLED (see 5A); teacher-only Style room SHIPPED 2026-10-04 (see 5B); students unchanged until the teacher releases it. Claudia's full design pass, written at the teacher's request ("lets brainstorm this Claudia, to get this feature completely developed in the dev plan before creating"). Nothing gets built until the teacher answers the Decisions list in section 5 (Decisions 1 to 6 block Phase 0).

Tracked in `DEVELOPMENT_PLAN.md`, Part B, "Style".

The teacher's request, verbatim

Kayden's own words (verbatim, kept close per Part C standard): "i want to redesign the players avatars completely, adding a format for the players around consistent animation (i.e., a frog and a bear both being biped characters with human-like animations such as a jump, walk (two legged), and conversation mouth) and a once-size fits all clothing library. We are going to call this "Style". I want to create a Sims CAS and dress-up game style catelogy/library of clothing that fits all characters for students and me (teacher), and for teacher/me only i want an ability to design clothing and accessories for these characters from templates (think blank/no pattern sewing patterns like t-shirt, pants, tank top) and adding patterns or designs. I want it to be like a POD (printify, for example) item creator after selecting from the templates. i also want the ability as a teacher to type what i want in an ai box for it to create. for example, if i selected a hat from the catelog, the ai box could be typed in about what i want the hat to look like, so i could write something like "Make a hat that is shaped like a triangle wedge of yellow swiss cheese." the ai should then generate the hat asset as described and then i can save to add it to catelog. In "style", as a teacher, iam -similar to POd websites- able to publish the items to marketplace for a defiend price, time period, achievement unlocj (ex. item unlocked after X number of questions were answered correctly in a specific game). lets brainstorm this Claudia, to get this feature completely developed in the dev plan before creating"

What exists today (verified in the repo, so the spec is grounded)

Verified in the repo this pass (so the spec is grounded, not guessed):

- `public/world/models/characters/player.glb` is a Kenney Mini Character ("character-male-a"). Its skeleton is exactly 7 bones: root, torso, head, arm-left, arm-right, leg-left, leg-right. It has 2 meshes (body-mesh, head-mesh) and a palette texture named "colormap" (a color swatch strip, NOT a painted UV unwrap).
- It already carries 32 animation clips: static, idle, walk, sprint, jump, fall, crouch, sit, drive, die, pick-up, emote-yes, emote-no, holding-right/left/both (+ shoot variants), attack-melee/kick left/right, interact-left/right, and 7 wheelchair-* clips. The game only ever plays idle and walk today. There is NO jaw/mouth bone, no wave, no talk, no cheer clip.
- Three separate copies of the player renderer exist: `PlayerModel` in `TownSquare.tsx`, plus separate player loaders in `HomeRoom.tsx` (line 223) and `IslandBuild.tsx` (line 375). The cake skin (`CharacterSkinOverlay`) only exists in Town Square.

- The Cake Character is a static, unrigged GLB (`public/world/character-skins/cake-character.glb`) bolted rigidly onto the `torso` bone, hiding body-mesh and head-mesh. Unlock: 100 Bakery Match correct answers. Stored in `students.unlockedCharacterIds` / `equippedCharacterId` , unlocked in `store.ts` (~line 1464).
- Two different things are both called "avatar": (1) the 2D portrait, 18 Kenney Blocky PNGs in `avatarCatalog.ts` (free starter, others \$20 to \$25 in Class Cash, owned in `ownedAvatarIds`), used at login, cards, Passport; (2) the 3D in-world character.
- `marketplace_items` already has kind, price (cents), `available_from/until` (date window), and `earnMethods` (purchase, daily-spin, bonus-spin, quest-reward, farmers-market). There is a cart, receipts, and `grantFreeMarketplaceItem` .
- Unlock counters today are ad hoc per-game student columns (`bakeryMilestoneCount` , `castleDefenseQuestionsAnswered` , plus `chess_games` rows). There is no generic "correct answers per game" counter.
- `api/` holds only `extract-article.ts` . Teacher auth is Supabase Auth. Students use the open anon key (schema.sql header says writes are open to anon, only DELETE is teacher-only).

1. Claudia's design take: why Style is right for this population, and where it can hurt

What is great about it

- **Autonomy and identity (SDT).** A dress-up character is the strongest autonomy lever in the whole game: no right answer, nothing to fail, fully the student's choice. For students who spend the day being told what to do, "this is me and I chose it" matters. It also serves competence (earning an item is visible proof of effort) and, if kept teacher-mediated, relatedness (a class "fashion show" the teacher runs).
- **Low floor, high ceiling.** A student who freezes at choices gets a Starter Look and a Surprise Me button and is done in two taps. A student who loves it can tint, layer, save six outfits, and chase earnable items.
- **The Studio is the novelty-decay answer.** Static badge shelves go stale by week 6. A teacher who can mint a new item in 10 minutes (the cheese hat, a class inside joke, a seasonal drop, an item named for a student's goal) renews the system herself. This is the "living system" the research favors.
- **Reinforcer menu that is individual.** Items can be released to selected students only, so an item can be a personal reward tied to a student's own goal (ABA: reinforcer preference, individualized, delivered immediately).
- **Representation.** One body-agnostic wardrobe with no gendered categories, plus always-free Comfort Gear (headphones/ear defenders, glasses, hearing aids, wheelchair, etc.) normalizes the tools this population uses.

Where it can hurt, and the built-in guard for each

- **Decision fatigue (ADHD/autism).** Guard: one category at a time, 6 big tiles per page, Starter Looks, Surprise Me (owned items only, always undoable), "Try it on" is free and instant, one primary button ("Save my look").
- **Predictability.** Guard: a student's look never changes by itself. No daily auto-rotation, no random re-skin, no item art that gets edited under them (published items are frozen).
- **Animation, sparkle and sound are features, not risks (teacher override, 2026-10-04).** Kayden: "sensory overload in terms of animations and music, sound effects is not a specific concern for the two

students i am creating this for. in fact, those are huge drivers of engagement and should be included. note for all future considerations" So Style INCLUDES animated items (sparkle, glitter, glow, shimmer, bounce, spin, trails, wiggling ears and tails), item sound effects (a jingle when a hat is equipped, squeaky shoes on walk, a cape whoosh on jump, a pop on try-on), and a celebratory sound and animation for unlocks and "Save my look". The only limit kept is photosensitive-seizure safety: no large-area flashing faster than 3 times per second. The existing per-student Less Motion setting (`worldReduceMotion`) still exists as an opt-in a teacher can switch on, never a default.

- **Status and comparison harm (A36).** Guard: no rarity tiers (common/rare/legendary), no "owned by N students," no "most popular," no countdown timers, no numbered scarcity ("only 10 made"). Students never see another student's ownership or progress. Rarity is a ranking mechanic in disguise.
- **Money anxiety and fairness.** Guard: every item has a free-earnable or teacher-grantable route; making an owned item cheaper or free prompts the teacher to refund owners (neurodivergent kids are often acutely sensitive to "that's not fair").
- **Purchase vs unlock (ABA).** Earned unlocks are stronger reinforcement of the target behavior (answering questions) than purchases. Recommendation: milestone items default to Earn only. Immediate delivery (within seconds, at a safe moment, see 6f) keeps the contingency clear.

2. Feature breakdown (named subsystems)

2a. The Homeplot Biped format (unified character + animation)

Honest starting point: Today's model is already a biped with a small shared rig, and every Kenney clip a frog and a bear need (idle, walk, sprint, jump, sit, crouch, emote-yes/no, interact, pick-up) already exists in `player.glb`. The "consistent animation" half of the request is largely free. What is missing: talk/mouth, wave, cheer, think, and real species bodies.

The rig contract ("Biped v1")

- Required bones, exact names, same bind pose: `root`, `torso`, `head`, `arm-left`, `arm-right`, `leg-left`, `leg-right` (the existing Kenney skeleton, reused on purpose so all 32 existing clips keep working and NPCs/neighbors, which appear to be the same Kenney family, can join the system later).
- Optional extra bones, never keyed by the shared clips, animated procedurally with a little sine/spring code: `ear-left`, `ear-right`, `tail`. (Clips that do not mention a bone simply leave it alone, so a bunny's ears or a frog's nothing do not break retargeting.)
- Named attachment sockets (empty child nodes): `socket-hat`, `socket-face`, `socket-back`, `socket-hand-left`, `socket-hand-right`.
- Proportion class: all species share bone lengths within about 10 percent. Species differ by MESH (head shape, belly, snout, ears), not skeleton. This is what makes one wardrobe fit all.
- Because the rig has no elbow, knee, or finger bones, every garment is a handful of rigid pieces parented to bones (see 2c). No skinning art problem. That is the single most important simplification in this plan.

Species v1 (recommended): Kid (human, wide skin-tone palette), Bear, Frog. Stretch fourth: Bunny (its ears stress-test hats, which is exactly why it is worth building second). Species is a free, changeable choice; outfit carries over when species changes.

Animation set v1 (shared by every species, named clips)

- Already exist, just need wiring: idle, walk, sprint (shown as "run"), jump, fall, sit, crouch, emote-yes, emote-no, interact-left/right, pick-up, die (not used in v1; never show a "dying" animation to students), holding-*
- New clips to author or generate once on the shared rig: **talk** (small head-and-arm gesturing loop), **wave**, **cheer** (for correct answers and unlocks), **think**, **shrug**, **dance** (optional).
- **Mouth / conversation:** no jaw bone exists. Recommended method: a face texture swap on the head mesh with 3 mouth frames (closed, small open, wide open) cycled while a "speaking" flag is true (NPC dialogue lines and, for the student, when text-to-speech is reading). Plus a slow blink on a timer. This works on the 7-bone rig, costs almost nothing on an iPad, and survives species with different snouts (each species supplies its own 3 face frames). A real jaw bone is the alternative but forces re-rigging every species.
- **Jump in Town Square:** an optional big on-screen Jump button (60px+, no keyboard, no hover). It is an expressive move, never required for anything.
- **Less Motion respects:** idle becomes nearly still, no bobbing, no auto-blink, mouth frames still change (that is communication, not decoration).
- **Wheelchair clips:** the existing `wheelchair-*` clips imply a wheelchair set is already in this pack. Offer it as an always-free, teacher-enabled "My Gear" item (see Decision 27), never priced.

Asset sourcing (the biggest risk, named plainly). Nobody has a free, CC0, rigged frog and bear biped that already matches this skeleton. Realistic routes:

1. **Build species procedurally in code (recommended first attempt).** Rounded boxes, spheres and capsules (head, snout, ears, eyes, belly) rigid-parented to the same 7 bones, with a shared palette. This matches the existing blocky Kenney look, needs no 3D artist, and I would expect it to pass a quick spike. It will look "toy-like," which is the aesthetic the world already has.
 2. **Commission species from a 3D artist** onto the existing 7-bone skeleton and clip set. Get quotes before committing a number. Needs: GLB, same bone names, 3 face frames, socket nodes, proper licensing (ideally work-for-hire, CC0 or full ownership).
 3. **Kit-bash from CC0 packs** (Quaternius, KayKit, Kenney). Most CC0 humanoid packs use a different (richer) rig, so reusing their clips means retargeting work. Not recommended for v1.
 4. **Text-to-3D for the base bodies. Do not.** Rig, topology and proportions will not match, and every clothing piece breaks.
- Phase 0 is a time-boxed spike specifically to answer this before anything else is committed.

2b. Style CAS (the dress-up screen, students and teacher)

Where it lives: one "Style" entry. Recommended: the Town Square pie menu's "My Stuff" area gets a "Style" button (icon plus label), and the Store building gets a "Style" tab in its sidebar. Same position on every screen that shows it. The old "Characters" tab becomes "Classic Portraits" (2D images only).

Navigation (max 6 top-level, one level deep): the category rail has 6 buttons, each icon plus text: **Body** (species, body color, face), **Head** (hats, hair, ears), **Face** (glasses, masks-free gear), **Clothes** (chips inside: Tops, Bottoms, Full-body, Shoes), **Extras** (back, held items), **Outfits** (saved looks). Always visible: Back and Home in the same corner every time (WebpageFrame pattern, matches Marketplace).

iPad layout

- Landscape: character stage on the left about 55 percent, category rail plus item grid on the right about 45 percent. Portrait: stage on top about 45 percent of height, category rail as a horizontal icon-plus-text row, grid below in 2 columns. Nothing relies on a wide desktop window.
- Item tiles 88px or larger, 12px or more spacing, 6 per page with big "Next page / Back page" buttons (no tiny scrolling grid).
- **Turntable:** one-finger horizontal drag on the stage only (touch-action: none on that canvas only, so page scroll still works elsewhere) PLUS big tap buttons "Turn left", "Turn right" and three view buttons "Front", "Side", "Back". Drag is never the only way.
- Stage renders on demand (not a constant loop) when idle, to save iPad battery and heat.

Flow and the one primary action

- Tap a tile = try it on instantly (preview only, nothing saved). Tapping an item you do not own still tries it on, and shows plain text under it: "Earn it: 37 of 100 questions" or "Add to cart: \$4.50". Try-before-you-want is a motivator (competence), never a dead end (matches the existing "you need \$X more, never a disabled button" rule).
- ONE primary button: **Save my look** (large, green, same spot). Secondary: **Put it back** (undo last), **Start over** (asks to confirm), **Surprise me** (random from OWNED items, undoable), **Cancel** (back to the saved look). Leaving with unsaved changes asks once: "Keep these changes? Yes / No".
- First-run for a student with no look: 3 screens, one decision each: pick your buddy (species), pick a color, pick a Starter Look. Then done.
- **Starter Looks:** 6 teacher-curated complete outfits, free, so a blank-slate freeze is never a thing.

Layering rules (plain, enforced, explained in kid words)

- One item per slot: hat, hair, face, top-base, top-outer (jacket over shirt, the only 2-layer slot), bottom, full-body, shoes, back, hand-left, hand-right.
- A full-body item (dress, costume) replaces top and bottom, and a small note says so: "The dress replaces your top and pants. Tap Put it back to undo."
- Each item declares what it hides (for example a beanie hides hair, ear tips poke through a hat hole, a hood hides hat) so nothing pokes through or double-stacks.
- Shoes fit under pants; skirts sit above the legs.

Color and tint: each item may have up to 2 tint zones. Students pick from 12 named, colorblind-safe swatches (name written under each swatch, never color-only). No free hex picker. Designs with fixed art (a printed cheese pattern) are marked not tintable by the teacher. "Original colors" button resets.

Saved outfits: 6 slots per student. Auto-generated thumbnail (offscreen render). Name chosen from a short picklist ("School", "Cozy", "Fancy", "Silly") or just a color dot, no free text in v1 (moderation and decision load). "Wear this" is always a manual tap; the game never swaps outfits by itself.

Teacher mode: same screen with every item unlocked (drafts included, labeled "Draft"), plus a "Show me what student X sees" switch. Per-student teacher controls (no code change): hide specific categories, limit visible item count, "Lock look" (for picture day or a student who needs it to stay stable).

2c. The one-size-fits-all clothing library

Slot taxonomy: Head (hat, hair, ears), Face (glasses, nose/cheek accessories, ear defenders), Top-base, Top-outer, Bottom, Full-body (dress, jumpsuit, costume like the cake), Shoes, Back (backpack, cape, wings), Hand (held item, left or right), Comfort Gear (cross-slot, always free, see rule below).

Two garment technologies (this is how fit actually works)

- 1. Shell garments (the main method).** A shirt is 3 rigid shells (one torso box, two sleeve boxes) parented to torso, arm-left, arm-right. Pants are 2 shells on the leg bones plus a waistband. Shoes are 2 shells on the leg bones. Each template owns its own UV layout (flat panels: front, back, sleeve, collar), so a printed design is just a texture on those panels. No dependency on the body mesh's UVs.
 - 2. Socket accessories.** Hats, glasses, backpacks, held items are rigid meshes on a socket node.
- Skirts, hoods, capes, sleeves that puff are all just shells that are not pretending to bend, which is fine because the character itself has no elbow or knee to bend.

How fit across species works

- Each species record carries `garment_fit` multipliers (torso, arm, leg scale) and `socket_offsets` (position, rotation, scale for each socket). A frog's wide head gets a larger hat scale and a forward glasses offset. A bear's round belly gets a slightly wider torso shell multiplier.
- Shells are authored about 5 percent larger than the base body so skin never pokes through. Each garment also declares `hides` (body parts to hide) to avoid z-fighting.
- Items carry a `fit_class` tag (normal, wide-head, tall-ears) so a hat that cannot work on a bunny is flagged in the Studio, not at the student.
- Perf: loose shells are fine for one player. For many characters on screen (NPCs wearing Style later) a one-time "bake outfit" step merges shells into one skinned mesh with one material, so each character costs 1 to 3 draw calls.

Launch library target (original or CC0, authored by the dev from Studio templates): about 60 items. Tops 8, bottoms 6, full-body 4, shoes 6, hats 10, hair 6, glasses 6, backpacks and capes 5, held items 6 (no weapons, ever), comfort gear 8, plus the Cake costume. All art original or CC0. No Pokemon/Minecraft/Webkinz-style lifted characters (checklist 8).

Comfort Gear rule (protected, per Part D regulation rule): ear defenders/headphones, glasses, hearing aids, wheelchair set, and any item a student could plausibly use as a sensory or access tool is ALWAYS free and is never sold, time-limited, or gated behind unlocks.

2d. Style Studio (teacher only, laptop)

Where: Teacher side, Game tab, next to Marketplace management. Laptop-first (the iPad-first rule governs student views; the Studio's outputs must still be checked on iPad). Behind `RequireTeacherAuth`.

Flow (3 steps, one clear step at a time):

- 1. Pick a template.** Gallery with thumbnails: Tops (tee, tank, long sleeve, hoodie, jacket), Bottoms (pants, shorts, skirt), Full-body (dress, jumpsuit), Shoes (sneaker, boot), Hats (cap, beanie, bucket, top hat, crown, party hat, wedge/cone/dome base shapes), Face (round and square glasses, ear defenders), Back (backpack, cape), Hand (blank flag, sign, balloon-like shapes). Blank means plain white, no pattern, like a sewing pattern.

2. Design.

- Left: a flat "pattern" editor showing the template panels (front, back, sleeves). Right: live 3D preview with Kid / Frog / Bear switch and turntable.
- Tools: color fill per zone; upload image (PNG/JPG/SVG, 5MB cap); place, scale, rotate, flip; layer order and opacity; repeat patterns (tile, half-drop, scale and spacing sliders) plus generators (stripes, dots, checks, gingham); text (curated fonts, teacher typed); undo/redo; "apply to all panels"; seam safe-area guide.
- Implementation note: a canvas editor library (for example Konva or Fabric.js) producing a layers JSON (saved, so the item stays re-editable) and a flattened texture PNG per template (512 to 1024px).

3. Details and save. Name, tags, slot, which zones are tintable, effects (see below), source label (Original / Pack name / AI), preview on all species, **Save as draft**. Nothing a student can see.

- **Effects (animation and sound):** pick from an effects shelf: sparkle, glitter, glow, shimmer, rainbow shift, bounce, spin, float, trail, plus a sound on equip, on walk, on jump, and on tap (from a built-in sound library or an uploaded clip). Effects preview live on the 3D model with sound. The only automatic check is photosensitive safety (no large-area flashing faster than 3 times per second), which blocks saving until fixed.
- **Honest limits:** this is "POD-style," not real print-on-demand. There is no Printify integration, no physical goods, no fulfilment. The teacher cannot invent new SHAPES in the designer; new templates are built by the dev (request via the plan). Shapes beyond templates come from the AI box (see 2e) or new dev-authored templates.

2e. The AI box (teacher only)

Hard truth: Claude cannot output images or 3D models, and the app has no generation service. This needs a third-party provider, called from a new Vercel function, with an API key the teacher creates and pays for.

What each approach can and cannot do

- **Option A: AI-made 2D art on an existing template (cheap, reliable, instant-ish).** A text-to-image model makes a pattern or print that is placed on the template's UV panels. CAN: "tiny yellow cheese slices and mice", "galaxy", "frog skin", "rainbow plaid", "sunflowers", a whole shirt graphic. CANNOT: change the SHAPE. Her own headline example ("shaped like a triangle wedge of yellow swiss cheese") would come back as a normal hat painted like cheese. Cost: typically cents per image (verify current pricing before committing). Quality: high and consistent. Extra work: make it seamless (offset-blend), remove backgrounds for stickers, and run a "Style normalizer" (downscale and palette-quantize to match the blocky world so AI art does not look pasted in).
- **Option B: Text-to-3D (Meshy, Tripo, Rodin-class APIs).** Generates a whole new GLB. CAN deliver a real cheese-wedge hat. CANNOT promise: consistent topology, clean pivot/scale/orientation, matching art style, no floating bits. Slower (tens of seconds to minutes), more cost per try (cents to a couple of dollars, verify), results vary run to run. Needs mesh simplification (aim 1 to 3k triangles), auto-fit, and still only works for rigid accessories (hat, glasses, back, held), never tops and bottoms.
- **Hybrid (recommended): "Shape + Paint."** Two tiers inside one AI box:
 - **Tier 1 (build first): parametric shape plus AI paint.** A language model (Claude via API, text only, which is something Claude can do) reads her prompt and returns a small JSON: base

shape from a dev-built family (dome, cone, wedge, cylinder, tall cylinder, flat brim, bowl, etc.), size, tint, and a texture prompt. The image model paints the texture. "Triangle wedge of yellow swiss cheese" becomes `{shape: wedge, size: medium, texture: "yellow swiss cheese surface with round holes"}`. Same topology every time, fits every species by construction, cheap, fast, iPad-light.

- **Tier 2 (later, spike-gated, optional): full text-to-3D** for truly one-off shapes. A "Try full 3D (slower, costs more)" button, budget-capped, accessory slots only, with a mandatory fit-and-simplify step in the teacher's browser. Do not build until Tier 1 is live and she says Tier 1 shapes were not enough.

Flow (all teacher-side):

1. In the Studio, pick a template (for example "Wedge hat").
2. Open the AI box, choose the mode chip: **Paint it** (texture only) or **Shape it** (accessory slots, parametric shape).
3. Type the prompt. A small "Safety check" runs first (see 2e Safety).
4. Press **Make it**; an estimated cost is shown first, a monthly spend meter is always visible.
5. Preview 1 to 4 variants on Kid / Frog / Bear.
6. **Try again**, **Tweak** (add words: "bigger holes", "more orange"), **Adjust fit** (nudge position/scale/rotation per species with sliders), **Edit in designer** (add text or stickers), or **Save as draft**.
7. Draft goes into the item list. Publishing is a separate step (2f).

Server design

- One Vercel function pair under `api/style/`: start-generation and check-status (async, so a slow job does not hit function time limits). It never returns the API key. It requires the teacher's Supabase session token and validates it server-side before spending any money (otherwise a student with dev tools could burn the teacher's budget, because students use the open anon key today).
- Heavy 3D processing (decimation, centering) happens in the teacher's BROWSER, not the function, to dodge serverless time and payload limits. Results upload to Supabase Storage.
- Provider and model are settings, not hard-coded, so a provider can be swapped.
- Monthly cap, generation count, and cost estimate live in `app_settings`; hard stop when cap is reached ("AI is paused for the month, you can still design by hand").
- Every generation is logged (prompt, provider, status, cost estimate, output paths).

Safety and moderation (students never see a prompt box, ever)

- Prompt gate before any generation: blocks weapons (the platform's zero-weapons rule applies to Style, including held items), violence, sexual content, hate, real brands and logos, real people, copyrighted characters, and anything on the teacher's own "avoid list" (for example spiders or clowns if a student is frightened by them).
- Output gate: provider safety filter, then an image review pass (a vision-capable model describes the render and flags the same categories), then the mandatory human step: the teacher sees the item on three species and must tick a publish checklist (no weapons, no brands/characters, nothing scary, text is correct, effects play correctly).
- **Text in AI images is often misspelled.** For a literacy-focused class this matters. The UI warns: "For words, use the Text tool." AI items with detected text get a visible warning.
- **Em dash sanitizer:** any AI-written item name or description is stripped of em dashes before it can be saved (language models produce them often, and the copy rule is a hard no).

- No student data (names, scores) is ever sent to any provider. Prompts are teacher-authored only.
- Provenance stored on every item: Original, Pack (name and license), AI (provider, model, date). Check each provider's terms for classroom and commercial use before enabling.

2f. Publishing to the Marketplace

Reuse, do not rebuild. Style items become a new `marketplace_items` kind, `style`, plus a link to the style item. That automatically inherits the cart, receipts, Count It Out checkout, the needs-vs-wants prompt, price overrides, date windows, `earnMethods` and the Daily Spin eligibility. No parallel store.

Publish dialog (one screen, one decision group at a time):

- **Price:** Class Cash in cents, \$0 allowed (free to claim).
- **How students get it** (pick one, plain labels): Buy it / Earn it / Earn it or buy it / Free for everyone / Teacher gives it.
- **Earn rule** (when Earn is chosen): metric plus threshold plus optional game. Metrics v1: correct answers in a chosen game (Bakery Match, Castle Defense, Slime Chess, Platformer, Gas Pump, any game), correct answers anywhere, questions attempted (effort, not accuracy), daily-streak days, assignments finished, activities completed, personal chess XP. Example from her words: "unlocked after 100 questions answered correctly in Bakery Match."
- **Counting:** from now vs lifetime total (Decision 14).
- **Time period:** From date, Until date, or always (existing date-window fields). Static text only on the student side ("Available until Oct 31"), never a countdown.
- **Audience:** All students, or selected students only (individual reinforcer).
- **Spin eligibility:** the existing earn-method chips (Daily Spin, Bonus Spin, Quest reward, Farmer's Market).
- **Return plan** (only for dated items): "Coming back" with an optional return date, or "Final run."
- **Preview** as a student would see the tile before pressing **Publish**.

Rules after publishing

- Published items are frozen. Name, price, and window can change; the art cannot (a student's favorite hat must not change under them). New art means "Save as new item."
- **Retire** hides it from the shop; owners keep it forever.
- Lowering price or making an owned item free asks: "N students paid \$X. Refund the difference to their Class Cash?" One tap. (Fairness for students sensitive to "that's not fair.")
- **Gifting:** teacher to student, one tap, uses the existing grant path. No student-to-student gifting in v1 (checklist 7, peer features must be teacher-mediated).
- **Limited editions:** seasonal drops yes. Manufactured scarcity ("only 10 exist"), rarity tiers, and "owned by N" no (A36 risk). A drop shelf shows what is coming and when, in plain text, so students can plan.

Unlock delivery (immediacy without interrupting work): when a rule is met, the unlock is queued and shown at a safe moment (end of the current question round, game over screen, or return to Town Square), never mid-question. The card says "New! You earned the Cheese Hat. Try it on?" with one big "Try it on" button and one "Later". The item appears in a "New" shelf in Style until seen. Student-facing progress ("37 of 100") is visible to that student only.

Counters: existing games keep their columns, but a new generic `student_counters` table (keys such as `correct:bakery`, `correct:castle`, `correct:chess`, `correct:any`, `attempts:any`) is added, backfilled from existing columns, and written through one `recordAnswer(studentId, gameId, correct)` call so future rules never need a new student column. Counters only ever go up (wrong never costs anything).

Teacher view: which students own which items, unlock progress per student, spend meter, and a per-student "give this item" action. Teacher-only; never shown to students.

2g. Data model (additions)

- `style_species` (id, name, glb_path or procedural_spec jsonb, face_frames jsonb, socket_offsets jsonb, garment_fit jsonb, palette jsonb, active).
- `style_templates` (id, slot, name, base_mesh_path or parametric_spec jsonb, uv_panels jsonb, tint_zones int, hides jsonb, thumb_path, active). Dev-authored only.
- `style_items` (id, template_id, slot, name, description, status draft|published|retired, source original|pack|ai-paint|ai-shape|ai-3d, design jsonb layers, texture_path, mesh_path nullable, fit jsonb per-species overrides, tintable_zones, hides, tags jsonb, effects jsonb (animation + sound settings), always_free bool, review jsonb {checked_by, checked_at, checklist}, ai_generation_id nullable, thumb_path, created_at). Insert/update restricted to the authenticated teacher by RLS (stricter than the app's usual open-write posture, because this is the catalog).
- `marketplace_items` gains: `style_item_id` text, `unlock_rule` jsonb ({methods: [{type:'purchase'|'achievement'|'free'|'grant', price_cents, metric, game_id, threshold, count_from:'lifetime'|'publish_date'}]}), `audience` jsonb (all or student ids), `return_plan` text, `return_date` date.
- `student_style_items` (student_id, style_item_id, acquired_via purchase|achievement|free|grant|migration, acquired_at, seen bool).
- `student_outfits` (id, student_id, slot_index 0 to 5, name_preset, equipped jsonb {slot: {item_id, tint: [c1,c2]}}, thumb_path, is_current bool). Plus student columns: `style_species_id`, `style_body_colors` jsonb, `style_prefs` jsonb (calm only, hidden categories, lock look). Teacher's own look in `app_settings.teacher_style` jsonb.
- `student_counters` (student_id, key, value, updated_at), primary key (student_id, key).
- `ai_generations` (id, item_id nullable, mode, prompt, provider, model, status, cost_cents_est, output_paths jsonb, moderation jsonb, created_at). `app_settings` gains `style_ai_enabled`, `style_ai_provider`, `style_ai_monthly_cap_cents`, `style_avoid_list` jsonb.
- **Storage buckets (public read, teacher write):** `style-art` (uploaded images, flattened textures), `style-models` (accessory GLBs), `style-thumbs` (item and outfit thumbnails). Same pattern as the existing `images` and `videos` buckets.
- **Env vars on Vercel (teacher provides keys):** one for the image provider, one for the language model, one for the optional 3D provider, plus the Supabase service key for the function to write Storage.
- Client-side unlock evaluation follows today's pattern (as the cake unlock does). Note: students use the open anon key, so a determined student with dev tools could self-grant; that is the platform's existing accepted risk, not new.

2h. How Style replaces or migrates today's avatars (A11) and the cake

- **In-world 3D character:** Style fully replaces `PlayerModel`, `CharacterSkinOverlay`, and the two other player loaders (Town Square, Home Room, Creative Island) with ONE shared `StyleAvatar` component, so all three scenes show the same look. This is also a cleanup of three duplicated renderers.
- **Everyone starts as "Kid" in the default Kenney look**, so no student's character suddenly changes on the day Phase 1 ships.
- **Cake Character:** becomes a **Full-body Costume item**, still earned at 100 Bakery Match correct answers (same rule, now expressed as an unlock rule). Existing `unlockedCharacterIds` and `equippedCharacterId` are migrated into `student_style_items` and the current outfit. Today it is a rigid mascot with no limbs. Recommended: rebuild it as a cake torso shell with the species' own arms and legs visible and animated, fixing the "arms out, no animation" problem the teacher flagged earlier. Needs her OK (Decision 7).
- **2D portraits (the 18 Blocky avatars, owned in `ownedAvatarIds`, some bought for \$20 to \$25):** never deleted, never taken away. They stay owned as "Classic Portraits." Recommended: new default portrait is an auto-rendered head-and-shoulders snapshot of the student's Style look, with a choice to keep the classic portrait instead. Login cards, chat, Passport, and Marketplace receipts use whichever the student picked.
- Marketplace sidebar: "Characters" tab is renamed "Classic Portraits" (existing price overrides keep working). New "Style" tab holds Style items.

3. Phased build order (each phase independently shippable)

Phase 0: Spike and decisions (nothing student-visible).

- In: confirm clip list (done), write the Biped v1 contract, spike procedural Frog and Bear (and Kid) on the 7-bone rig; test shell T-shirt, pants, hat, glasses on all three; iPad performance test (one player plus about 10 NPCs with outfits, Safari, both orientations); decide procedural vs commissioned bodies; get the teacher's answers to Decisions 1 to 6.
- Out: any UI. Dependency: teacher answers. Gate: go or no-go on the art path.

Phase 1: One avatar, many species (no clothing yet).

- In: shared `StyleAvatar` replacing the three player renderers; species picker (Kid, Bear, Frog) with the 3-screen first-run; body color; mouth frames and talk, wave, cheer clips; optional Jump button; Cake kept working as today (temporary adapter).
- Out: clothing, Marketplace, Studio, AI. Dependency: Phase 0. Ships alone: yes.

Phase 2: Style CAS plus starter library (all free).

- In: category rail, try-on, Save my look, Put it back, Surprise me, 6 saved outfits, Starter Looks, tints, Comfort Gear, about 40 dev-authored items, Cake migrated to a Costume item, teacher mode, per-student Calm/lock settings, teacher's own character, headshot portraits, iPad pass in both orientations.
- Out: buying, earning, Studio. Dependency: Phase 1. Ships alone: yes.

Phase 3: Marketplace and unlock rules.

- In: `style` marketplace kind, publish dialog (price, window, audience, earn rules, return plan), `student_counters` plus backfill, `recordAnswer`, deferred unlock cards, "New" shelf, teacher grants,

refund-on-price-drop prompt, Daily Spin eligibility for style items, Receipts. Teacher can already publish the dev's library items.

- Out: designing new items. Dependency: Phase 2. Ships alone: yes.

Phase 4: Style Studio (hand design, no AI).

- In: template gallery, flat pattern editor, upload, patterns, text, effects shelf (animation and sound), preview on three species, drafts, publish checklist, provenance. About 20 templates.
- Out: AI. Dependency: Phase 2 (library format) and Phase 3 (to publish). Ships alone: yes.

Phase 5: AI Tier 1 (Paint it, and Shape it with parametric shapes).

- In: `api/style/*` functions with teacher-token verification, provider settings, monthly cap, cost estimate, prompt gate, output moderation, em dash sanitizer, text-misspelling warning, regeneration/tweak, fit sliders, generation log.
- Out: true text-to-3D. Dependency: Phase 4 and Decisions 21 to 25. Ships alone: yes.

Phase 6 (optional, spike-gated): AI Tier 2 text-to-3D accessories.

- In: provider integration, in-browser simplify and auto-fit, per-species fit sliders, tighter budget cap. Only if she says Tier 1 shapes were not enough.

Phase 7: Extensions (each needs her go-ahead). NPCs and Neighbors wearing Style items (the 4 Neighbors plus ambient townspeople), "Dress for the Day" independence activity, seasonal drop calendar, more species (Bunny first), pet accessories, a teacher-run "Fashion Show" class screen.

4. Risks and honest limits

- 1. Species assets are the biggest risk.** No off-the-shelf rigged frog and bear match the skeleton. Procedural bodies are the cheapest path; if they look wrong, a commission is needed (get quotes first). Budget and calendar both hinge on Phase 0.
- 2. Body UVs.** Today's models use a color-swatch palette texture, not a paintable unwrap. Painting directly onto bodies would need re-authored meshes. Shell garments with their own UVs avoid this entirely, which is why they are the recommended method.
- 3. Rigid garments do not flex.** A dress or hoodie cannot swing or crease like Sims cloth. With a 7-bone, elbowless rig this is consistent, but it limits the look. Needs her expectations set early.
- 4. AI 3D quality.** Text-to-3D output is unpredictable, can fail, and will not match the world's art style without work. Do not promise it. Tier 1 covers most of what she described (the cheese wedge becomes a parametric wedge with cheese paint).
- 5. AI cost and key handling.** Teacher pays per generation. Needs a cap, a visible meter, and a server that checks the teacher's login so students cannot spend her money.
- 6. iPad performance.** Many skinned characters, extra draw calls from shells, and large textures can cause thermal throttling or Safari memory loss. Mitigations: bake outfits, 512px textures, only equipped items loaded, on-demand stage rendering, limit animated characters in view, test on a real iPad.
- 7. Moderation and IP.** AI can produce brands, characters, scary or off-model content, and garbled text. Gates plus mandatory teacher review. Her uploads need rights (some bought asset packs forbid reuse in derived art; check each license).

8. **Scope.** This is a multi-month program, not a feature. Phases 1 to 3 deliver the heart (identity plus earning) without any AI.
9. **Existing student investment.** Students may have spent Class Cash on 2D portraits. They must be kept.
10. **Cheating surface.** Open anon writes mean ownership rows can be forged by a determined student. Same as existing economy; flagged, not new.

5. Decisions only the teacher can make (each with Claudia's recommended default)

1. **Art direction.** Stay with the world's blocky low-poly Kenney look, or move to something smoother?
Default: stay blocky. Smoother needs a commission and different clothes.
2. **Species for launch.** Default: Kid, Bear, Frog. Bunny is the first add. Any other animal you want first?
3. **How we get the species.** Default: try procedural bodies in code (no cost); only commission if the spike fails. What is your ceiling if a commission is needed?
4. **Does Style replace A11's 3D characters completely?** Default: yes for the in-world character. Is that right?
5. **Portrait (login cards, chat, Passport).** Default: auto head snapshot of the Style look, with an option to keep the classic image. OK?
6. **The 18 Blocky portraits.** Default: every student keeps what they own, they remain selectable as "Classic Portraits," and we stop adding new ones. Keep selling the unowned ones at current prices, or stop?
7. **Cake Character.** Default: becomes a Full-body Costume item with the species' arms and legs visible and animated, same 100 Bakery Match unlock, owners keep it. OK to rebuild it that way instead of the current rigid mascot?
8. **Can students recolor items?** Default: yes, 2 zones, 12 named swatches, not on fixed-art items.
9. **Body colors and skin tones.** Default: yes, a curated palette per species, plus a wide human skin-tone set for Kid.
10. **Saved outfits per student.** Default: 6.
11. **Free starter set.** Default: every student gets about 15 free items across slots, 6 Starter Looks, and all Comfort Gear.
12. **Comfort Gear always free, never time-limited.** Default: yes (protected rule, ties to the regulation principle).
13. **Can an achievement item also be bought?** Default: you choose per item. Milestone items default to Earn only so the achievement means something. You can always give any item to a student by hand (for a student the earn rule is unfair to).
14. **What counts toward unlocks?** Default: correct answers, with an "attempts" option for effort-based items, counted lifetime (students with past work unlock right away). Or count from publish date?
15. **Badge pause.** Badges and auto-awards are paused platform-wide (`BADGES_PAUSED`). Default: Style unlock rules are a separate system and ARE allowed because you asked for them. Confirm that is not what you meant to pause.
16. **Time-limited items.** Default: allowed, shown as plain dates, no countdowns, minimum 14 days, and each one is marked "Coming back" or "Final run." Owners always keep it. Any students for whom you want these hidden entirely (a per-student "no limited items" switch)?
17. **Items for selected students only.** Default: yes, as individual reinforcers tied to a goal.

18. **Gifting.** Default: you can gift to students. No student-to-student gifts in v1.
19. **Your own character.** Default: yes. Where should it appear: Teacher Home header, chat avatar, a "Teacher" NPC in Town Square, class screens?
20. **Where students find Style.** Default: a Style button in the pie menu and a Style tab in the Store, same icon and label in both. OK?
21. **AI providers and budget.** Default: one image provider plus Claude's API for reading the prompt and checking safety, monthly cap of \$10, hard stop. Which providers are you willing to make accounts with, and what monthly cap is right?
22. **True text-to-3D.** Default: skip for now. Are parametric shapes (wedge, cone, dome, cylinder and so on, painted by AI) good enough for the cheese hat, or is a fully custom shape a must-have?
23. **AI avoid list.** Default: zero weapons, no brands, no real people, no copyrighted characters, nothing scary, plus your own list. What should go on yours?
24. **Review before publish.** Default: nothing ever auto-publishes; you tick a checklist on every item. OK?
25. **Image rights.** Which sources are you allowed to upload from (your own art, purchased packs)? Default: the Studio records the source on every item and warns on unknown ones.
26. **Student-made designs.** Default: not in v1. Possible later as a palette-only "Pattern Painter" that you approve before use.
27. **Wheelchair and access gear.** Default: offer the wheelchair set, hearing aids, glasses, and ear defenders as always-free Comfort Gear, enabled per student by you. OK?
28. **Mouth style.** Default: 3-frame face swap (cheap, works on iPad). The alternative is a real jaw bone, which means re-rigging every species.
29. **Jump.** Default: an optional big Jump button in Town Square, expressive only. Wanted?
30. **Can students ever see each other's looks?** Default: no. Town Square is one student per device. A class "Fashion Show" would be run by you on one screen.
31. **Teacher analytics.** Default: you see who owns what and unlock progress, never shown to students.
32. **Edits after publish.** Default: art is frozen; name, price, and window can change; new art is a new item.
33. **Price drops.** Default: a prompt offers a refund of the difference to anyone who already paid more.
34. **Naming.** "Style" for the feature and the button. Confirm the label and a single icon to pair with it.

5A. The teacher's answers (2026-10-04): SETTLED, do not revisit without new instruction

Kayden answered the list in one message ("lets get to wrok"). Her words are paraphrased only where she answered with a number; quotes are verbatim.

- **Assets (her lead note):** "the assets are a main thing here but im having a hard time finding." She then shared 8 Sketchfab models as the look she wants (cute, rounded, big-headed cartoon animals): a cartoon "Animal Friends" dog, a Dalmatian (character209), a dog, a cartoon Bengal cat, a stylized cute cartoon cat, a frog (character156), a rigged cartoon frog, and a cartoon capybara. Links are kept below in "Reference models". Sketchfab is blocked from this build environment, so their licenses and rigs could not be checked; they are a style target, not downloaded assets.
- **1. Art direction:** match those references: cute cartoon, chibi proportions (big head, big shiny eyes, soft rounded body).
- **2. Launch species: dog, cat, frog, capybara.**
- **3. How:** "try procedural bodies in code; keep the same body movements for all, same general template so the clothes are one size."

- **4. Style replaces A11's 3D characters completely.**
- **5.** Yes: the student's portrait becomes their Style character.
- **6 and 7.** "this should overhaul and delete all previous avatars and portraits. replace with new, styled character." and "delete all old avatars, characters." Supersedes Claudia's "grandfather Classic Portraits" default: the 18 Blocky portraits, the 3D player skin and the Cake Character are all removed **when Style is released to students** (not before; see 20).
- **8.** Students recolor with a **color wheel drag selector**, and pick **textures/patterns (plaid, polka dots...)** and **change the colors inside those too.**
- **9.** "they can change any and all colors with color wheel selector" (body colors included, no curated-only palette).
- **10. One outfit at a time. No saved outfits.**
- **11.** Free starter set: yes.
- **12.** Comfort Gear always free and never time-limited: yes.
- **13.** Achievement items can NOT also be bought.
- **14.** Correct answers count **from the publish date** (not lifetime).
- **15.** Badges stay paused (`BADGES_PAUSED` unchanged).
- **16.** Time-limited items: yes.
- **17.** Selected-students-only items: not needed.
- **18.** Teacher gifting: yes.
- **19.** The teacher's own character: "yes!!"
- **20.** "students should not have access to style until i explicitly say its ready to send to students (for now keep everything as is for them), but when i say so, style should be selected in their main pie menu."
- **21.** AI budget: "\$0 outside of regular monthly payment from subscription." See "AI box under a \$0 budget" below.
- **22.** The cheese hat "should look like a holey triangle of cheese."
- **23.** Avoid list: "zero weapons or anything scary. keep whimsical, silly, and fun for middle schoolers/upper elementary."
- **24.** Review before publish: yes, plus an **auto-save draft** feature in the Studio.
- **25.** Image rights: "this is personal use only, i will vet everything i send to you so its safe to use."
- **26.** Student-made designs: yes, later, as a full **Seamstress game** (see below).
- **27.** Wheelchair and access gear as Comfort Gear: "love yes."
- **28.** Mouth movement is required ("mouth movements are necessary").
- **29.** Jump: yes ("jump is just fun for kids").
- **30.** Students seeing each other: "town square will be live for both students eventually, where they can see each other, but for now its not."
- **31.** Teacher analytics: sure.
- **32. Teacher edits after publish DO update for students**, even for items they already own (supersedes Claudia's "published art is frozen" default).
- **33.** Price-drop refunds: sure. Plus a new idea: **sales, including daily sales flyers** for the Marketplace.
- **34.** Name "Style": yes.
- **Sensory (2026-10-04, separate message):** animation, music and sound effects are engagement drivers for her two students, not risks (Part D standing override).

AI box under a \$0 budget (consequence of answer 21). Every text-to-image or text-to-3D service, and Claude's own API, bills per use outside a Claude subscription, so the app cannot call any AI service at runtime without new cost. Plan instead:

1. **Free in-app "Describe it" builder:** the AI box reads the teacher's words with a built-in word list (shapes like wedge, cone, dome, tall hat; colors; patterns like stripes, polka dots, plaid; words like holes, spots, stars) and builds the item from the parametric shapes and patterns already in the wardrobe code. Instant, free, limited to what the parts kit can make.
2. **"Ask Claude to build it" requests:** anything the builder can't make is saved as a request (her exact words) to a list Claude works through in a Claude Code session, building it as real code like the Swiss Cheese Hat. This uses her subscription, not a paid API.
3. A paid image or 3D service stays parked unless she later decides to pay for one.

Reference models (her picks, 2026-10-04):

- <https://sketchfab.com/3d-models/animal-friends-cartoon-dog-3ca469323c8641018537b6873303244d>
- <https://sketchfab.com/3d-models/character209-dalmatian-dog-6727c6c5bc4f4a68b9681bc61bdd55ee>
- <https://sketchfab.com/3d-models/dog-3df1138f25d348b8b5c662439b00d436>
- <https://sketchfab.com/3d-models/cartoon-bengal-cat-0cb275d5b70842faadaabe5e8232a73f>
- <https://sketchfab.com/3d-models/stylized-cartoon-cat-character-cute-3d-model-26fc00d185cc49e08ad5ac8fbf21db07>
- <https://sketchfab.com/3d-models/character156-frog-0cad7665061543a78178bd9228addbe8>
- <https://sketchfab.com/3d-models/asset-cartoons-character-animals-frog-rig-de9e52f044fe4bc3a0be27b79a914da1>
- <https://sketchfab.com/3d-models/capybara-carton-49a344dcb02b4b14a065f39e7fb8708f> If she wants any of these exact models in the game, she can download them (checking each one's license) and send the files; each would still need refitting to the shared body template so one-size clothing keeps working.

Future: the Seamstress game with Webkinz-style clothing recipes (direct teacher idea, 2026-10-04).

"eventually i want to have a whole seamstress game where they can design their own clothes similar to the style designer i have as a teacher, though with questions intermittently interrupted from question sets." Then, with two Webkinz screenshots ("Clothing Recipe Revealed!" via a Clothing Machine: three items in, one new item out; and "Fashion Week Recipes," rows of item + item + item = new item): "when students have access and we are working on building the seamstress game, i want a webkinz clothing recipe inspired activity for them to do and then have their own custom clothes to wear." Planned shape (to design with Claudia when Seamstress starts): a **Clothing Machine** where students combine 3 items they own (or fabric swatches/patterns) to discover a new item from a recipe book; discovered recipes are logged in a Recipe Book (collect them all); new items go straight into their wardrobe to wear; the machine runs on question-set questions between steps, same gate pattern as the other native games; the student's own design tool is a simplified version of the teacher Studio (pattern + colors on templates). Status: idea recorded, not designed or built; it follows student release.

Other ideas from this message, recorded: Town Square going live with both students seeing each other (future multiplayer, not now); Marketplace sales and daily sales flyers (future).

5B. Build status

Shipped 2026-10-04 (teacher-only, students see no change):

- `src/style/`: the shared body template (`species.ts`), the four species built in code (`StyleCharacter.tsx`: dog with floppy ears and a wagging tail, cat with pointed ears, whiskers and a striped coat, frog with eyes on top and spots, capybara with a long blunt snout and tiny ears), chibi proportions, big shiny eyes that blink, and shared animations: stand, walk, run, jump, wave, cheer, dance, plus a talking mouth that opens and closes.
- One-size wardrobe (`wardrobe.tsx` , 27 items): T-Shirt, Tank Top, Long Sleeve, Hoodie, Dress; Pants, Shorts, Skirt; Sneakers, Boots, Rain Boots; Ball Cap, Beanie, Party Hat, Crown, Top Hat, Bucket Hat, **Swiss Cheese Hat** (a holey triangle wedge with holes all the way through and dents on top); Round Glasses, Square Glasses, Star Shades, Heart Shades; Comfort: Ear Defenders, Headphones; Back: Backpack, Hero Cape (sways), Butterfly Wings (flap).
- 10 patterns (Solid, Polka Dots, Stripes, Plaid, Gingham, Checks, Stars, Hearts, Zigzag, Spots), each with both colors editable, on every clothing zone AND on the animal's fur, tummy and ears/paws.
- Drag color wheel with brightness bar and quick colors (`ColorWheel.tsx`).
- The **Style** room at Teacher nav → 🐼 Style (`/teacher/style`): pick the animal, color everything, wear one outfit (one item per slot), Surprise me, Start over, Undo changes, Save my look, turn buttons plus drag to spin, animation buttons, sound effects on every action (`styleSounds.ts`), and a cheer when saved.
- Saving: the teacher's look saves to a new `style_looks` table (needs the SQL in `supabase/schema.sql`) and to the device as a backup.

Next (in order): students' version + release switch (pie menu entry, replaces the 3D player in Town Square/Home Room/Island, portraits become Style snapshots, old avatars/characters deleted, all at release); Marketplace publishing with price, dates, earn rules counted from publish date, gifting, refunds, sales; Style Studio (templates, POD-style designer, effects shelf, auto-save drafts) with the free Describe-it builder and the request list; more items and the Wheelchair set in Comfort Gear; Seamstress game later.

6. Conflicts with existing standing rules and shipped features (named explicitly)

1. **A11 and A12 two avatar systems.** The plan describes avatars as one catalog; reality is 2D portraits plus 3D skins. Style must define both, and paid portraits must be grandfathered.
2. **Her earlier cake instruction** ("ensure it uses the human player assets for movement and animations. it cant just have its arms out"): the current cake is a rigid mascot because the source mesh has no limbs. Style supersedes that workaround; the Costume rebuild needs her OK.
3. **Three duplicated player renderers** (TownSquare, HomeRoom, IslandBuild) and the NPC loaders: must be unified or Style will show in one scene and not the others.
4. **A36 No Leaderboards.** Style must add no rarity, no ownership counts, no "popular," no cross-student view. Using chess XP or answer counts as an unlock metric is fine because it shows only that student's own progress (consistent with A47's personal-only XP).
5. **BADGES_PAUSED.** Achievement unlocks sound like badges. They are a separate system but conflict with the spirit of "don't add any without me saying it." Decision 15.
6. **Regulation path never gated behind currency (Part D).** Comfort Gear must never be purchasable-only, time-limited, or earn-gated.
7. **"Wrong never costs anything."** Counters never decrement. Correct-only metrics can disadvantage students who struggle with accuracy; offer attempts-based metrics too (checklist 10, reward effort).

8. **Zero-weapons rule (Part D).** Currently enforced only in the Build Mode catalog; must be extended to held items and AI prompts.
9. **Marketplace "Settled, won't build" list** (ratings, reviews, "customers also bought") excludes manipulative purchase pressure. Countdown timers, scarcity numbers and rarity are the same family and are excluded here. Dated windows already exist, so they stay, minus the pressure cues.
10. **Class Cash is a real-money economy (A30).** Prices stay in cents and use the existing cart and Count It Out checkout. AI generation cost is the teacher's real money, never Class Cash.
11. **Daily Spin (A13).** The spin pool must learn the `style` kind, still deterministic per date, with the existing "already owned" cash fallback.
12. **Open Supabase RLS.** The catalog tables should be teacher-write-only, and the AI function must validate the teacher session. Both are stricter than most of the current schema, deliberately.
13. **Original IP (checklist 8).** AI and uploads are a new path for lifted characters and brands; the prompt gate, output review, and provenance field exist for this.
14. **No em dashes.** AI-written names and descriptions need an automatic sanitizer.
15. **No pop-ups interrupting a task.** Unlock cards are deferred to safe moments.
16. **iPad-first.** Turntable drag must have tap alternatives and be tested with real touch-drag; no hover; 44px minimum (this spec uses 88px item tiles and 60px primary buttons); both orientations; no keyboard shortcuts. The Studio is laptop-only, but every item it produces must be verified in the iPad CAS.
17. **Only one serverless function exists today** (`extract-article.ts`), and no secrets pattern is documented. The first paid-API function is a new security surface.
18. **Dev plan standing rule.** Each phase's push to `main` must update the plan (shipped, in progress, ideas) in the same push.

7. Standing rules this feature adds (also recorded in DEVELOPMENT_PLAN.md Part D)

- **Comfort Gear is always free.** Any Style item that works as a sensory or access tool (ear defenders, glasses, hearing aids, wheelchair set) is never sold, time-limited, or earn-gated.
- **No manufactured scarcity or status in any shop.** No rarity tiers, no "owned by N," no countdown timers, no numbered editions.
- **No weapons anywhere in Style**, including held items and AI prompts.
- **Students never see a generation or prompt box.** All AI features are teacher-only, teacher-reviewed, spend-capped, and logged.
- **A student's look changes only when that student changes it** (or when the teacher uses Lock look for them). Published item art is frozen.
- **Unlock moments are queued to safe moments**, never mid-question.